

Algorithms

Do not multiply entities beyond necessity.
– William Occam

- Introduction
- Use of Iterators
- Iterator Types
- Stream Iterators
- Predicates
- Algorithm Overview
- Container Algorithms
- Advice

10.1 Introduction

A data structure, such as a list or a vector, is not very useful on its own. To use one, we need operations for basic access such as adding and removing elements (as is provided for **list** and **vector**). Furthermore, we rarely just store objects in a container. We sort them, print them, extract subsets, remove elements, search for objects, etc. Consequently, the standard library provides the most common algorithms for containers in addition to providing the most common container types. For example, the we can simply and efficiently sort a **vector** of **Entry**s and place a copy of each unique **vector** element on a **list**:

```
void f(vector<Entry>& vec, list<Entry>& lst)
{
    sort(vec.begin(),vec.end());           // use < for order
    unique_copy(vec.begin(),vec.end(),lst.begin()); // don't copy adjacent equal elements
}
```

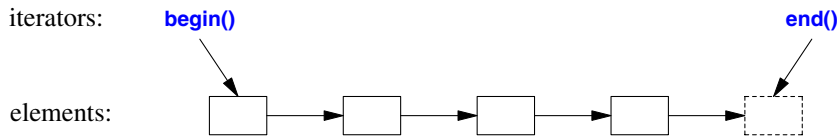
For this to work, less than (<) must be defined for **Entry**s. For example:

```

bool operator<(const Entry& x, const Entry& y)    // less than
{
    return x.name<y.name;                      // order Entrys by their names
}

```

A standard algorithm is expressed in terms of (half-open) sequences of elements. A *sequence* is represented by a pair of iterators specifying the first element and the one-beyond-the-last element:



In the example, `sort()` sorts the sequence defined by the pair of iterators `vec.begin()` and `vec.end()` – which just happens to be all the elements of a `vector`. For writing (output), you need only to specify the first element to be written. If more than one element is written, the elements following that initial element will be overwritten. Thus, to avoid errors, `lst` must have at least as many elements as there are unique values in `vec`.

If we wanted to place the unique elements in a new container, we could have written:

```

list<Entry> f(vector<Entry>& vec)
{
    list<Entry> res;
    sort(vec.begin(),vec.end());
    unique_copy(vec.begin(),vec.end(),back_inserter(res));    // append to res
    return res;
}

```

The call `back_inserter(res)` constructs an iterator for `res` that adds elements at the end of a container, extending the container to make room for them. This saves us from first having to allocate a fixed amount of space and then filling it. Thus, the standard containers plus `back_inserter()`s eliminate the need to use error-prone, explicit C-style memory management using `realloc()`. The standard-library `list` has a move constructor (§4.6.2) that makes returning `res` by value efficient (even for `lists` of thousands of elements).

If you find the pair-of-iterators style of code, such as `sort(vec.begin(),vec.end())`, tedious, you can define container versions of the algorithms and write `sort(vec)` (§10.7).

10.2 Use of Iterators

When you first encounter a container, a few iterators referring to useful elements can be obtained; `begin()` and `end()` are the best examples of this. In addition, many algorithms return iterators. For example, the standard algorithm `find` looks for a value in a sequence and returns an iterator to the element found:

```

bool has_c(const string& s, char c)    // does s contain the character c?
{
    auto p = find(s.begin(),s.end(),c);
    if (p!=s.end())
        return true;
    else
        return false;
}

```

Like many standard-library search algorithms, `find` returns `end()` to indicate “not found.” An equivalent, shorter, definition of `has_c()` is:

```

bool has_c(const string& s, char c)    // does s contain the character c?
{
    return find(s.begin(),s.end(),c)!=s.end();
}

```

A more interesting exercise would be to find the location of all occurrences of a character in a string. We can return the set of occurrences as a `vector` of `string` iterators. Returning a `vector` is efficient because `vector` provides move semantics (§4.6.1). Assuming that we would like to modify the locations found, we pass a non-`const` string:

```

vector<string::iterator> find_all(string& s, char c)    // find all occurrences of c in s
{
    vector<string::iterator> res;
    for (auto p = s.begin(); p!=s.end(); ++p)
        if (*p==c)
            res.push_back(p);
    return res;
}

```

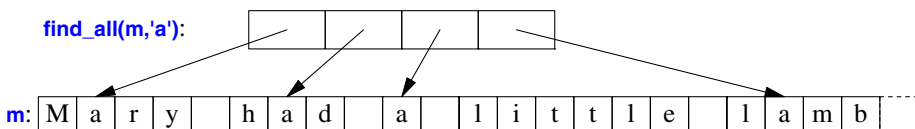
We iterate through the string using a conventional loop, moving the iterator `p` forward one element at a time using `++` and looking at the elements using the dereference operator `*`. We could test `find_all()` like this:

```

void test()
{
    string m {"Mary had a little lamb"};
    for (auto p : find_all(m,'a'))
        if (*p!='a')
            cerr << "a bug!\n";
}

```

That call of `find_all()` could be graphically represented like this:



Iterators and standard algorithms work equivalently on every standard container for which their use makes sense. Consequently, we could generalize `find_all()`:

```
template<typename C, typename V>
vector<typename C::iterator> find_all(C& c, V v)           // find all occurrences of v in c
{
    vector<typename C::iterator> res;
    for (auto p = c.begin(); p!=c.end(); ++p)
        if (*p==v)
            res.push_back(p);
    return res;
}
```

The `typename` is needed to inform the compiler that `C`'s `iterator` is supposed to be a type and not a value of some type, say, the integer 7. We can hide this implementation detail by introducing a type alias (§5.7) for `Iterator`:

```
template<typename T>
using Iterator = typename T::iterator;           // T's iterator

template<typename C, typename V>
vector<Iterator<C>> find_all(C& c, V v)           // find all occurrences of v in c
{
    vector<Iterator<C>> res;
    for (auto p = c.begin(); p!=c.end(); ++p)
        if (*p==v)
            res.push_back(p);
    return res;
}
```

We can now write:

```
void test()
{
    string m {"Mary had a little lamb"};

    for (auto p : find_all(m,'a'))                // p is a string::iterator
        if (*p!='a')
            cerr << "string bug!\n";

    list<double> ld {1.1, 2.2, 3.3, 1.1};
    for (auto p : find_all(ld,1.1))
        if (*p!=1.1)
            cerr << "list bug!\n";

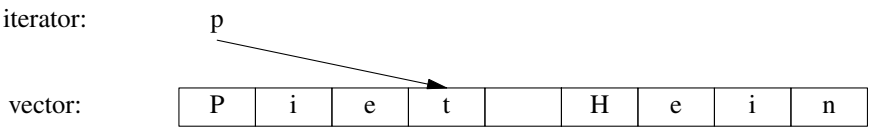
    vector<string> vs { "red", "blue", "green", "green", "orange", "green" };
    for (auto p : find_all(vs,"red"))
        if (*p!="red")
            cerr << "vector bug!\n";
}
```

```
for (auto p : find_all(vs,"green"))
    *p = "vert";
}
```

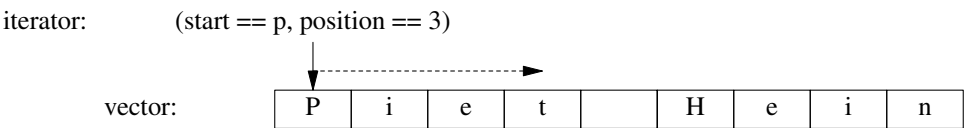
Iterators are used to separate algorithms and containers. An algorithm operates on its data through iterators and knows nothing about the container in which the elements are stored. Conversely, a container knows nothing about the algorithms operating on its elements; all it does is to supply iterators upon request (e.g., `begin()` and `end()`). This model of separation between data storage and algorithm delivers very general and flexible software.

10.3 Iterator Types

What are iterators really? Any particular iterator is an object of some type. There are, however, many different iterator types, because an iterator needs to hold the information necessary for doing its job for a particular container type. These iterator types can be as different as the containers and the specialized needs they serve. For example, a `vector`'s iterator could be an ordinary pointer, because a pointer is quite a reasonable way of referring to an element of a `vector`:

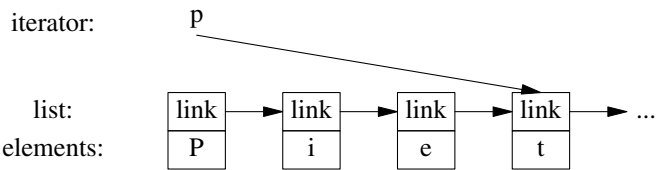


Alternatively, a `vector` iterator could be implemented as a pointer to the `vector` plus an index:



Using such an iterator would allow range checking.

A `list` iterator must be something more complicated than a simple pointer to an element because an element of a `list` in general does not know where the next element of that `list` is. Thus, a `list` iterator might be a pointer to a link:



What is common for all iterators is their semantics and the naming of their operations. For example, applying `++` to any iterator yields an iterator that refers to the next element. Similarly, `*` yields

the element to which the iterator refers. In fact, any object that obeys a few simple rules like these is an iterator – *Iterator* is a concept (§5.4). Furthermore, users rarely need to know the type of a specific iterator; each container “knows” its iterator types and makes them available under the conventional names `iterator` and `const_iterator`. For example, `list<Entry>::iterator` is the general iterator type for `list<Entry>`. We rarely have to worry about the details of how that type is defined.

10.4 Stream Iterators

Iterators are a general and useful concept for dealing with sequences of elements in containers. However, containers are not the only place where we find sequences of elements. For example, an input stream produces a sequence of values, and we write a sequence of values to an output stream. Consequently, the notion of iterators can be usefully applied to input and output.

To make an `ostream_iterator`, we need to specify which stream will be used and the type of objects written to it. For example:

```
ostream_iterator<string> oo {cout};    // write strings to cout
```

The effect of assigning to `*oo` is to write the assigned value to `cout`. For example:

```
int main()
{
    *oo = "Hello, ";    // meaning cout<<"Hello, "
    ++oo;
    *oo = "world!\n";   // meaning cout<<"world!\n"
}
```

This is yet another way of writing the canonical message to standard output. The `++oo` is done to mimic writing into an array through a pointer.

Similarly, an `istream_iterator` is something that allows us to treat an input stream as a read-only container. Again, we must specify the stream to be used and the type of values expected:

```
istream_iterator<string> ii {cin};
```

Input iterators are used in pairs representing a sequence, so we must provide an `istream_iterator` to indicate the end of input. This is the default `istream_iterator`:

```
istream_iterator<string> eos {};
```

Typically, `istream_iterators` and `ostream_iterators` are not used directly. Instead, they are provided as arguments to algorithms. For example, we can write a simple program to read a file, sort the words read, eliminate duplicates, and write the result to another file:

```
int main()
{
    string from, to;
    cin >> from >> to;                // get source and target file names

    ifstream is {from};                // input stream for file "from"
    istream_iterator<string> ii {is};   // input iterator for stream
    istream_iterator<string> eos {};    // input sentinel
```

```

    ofstream os {to};           // output stream for file "to"
    ostream_iterator<string> oo {os, "\n"}; // output iterator for stream

    vector<string> b {ii, eos};   // b is a vector initialized from input
    sort(b.begin(), b.end());    // sort the buffer

    unique_copy(b.begin(), b.end(), oo); // copy buffer to output, discard replicated values

    return !is.eof() || !os;      // return error state (§1.3, §8.4)
}

```

An **ifstream** is an **istream** that can be attached to a file, and an **ofstream** is an **ostream** that can be attached to a file (§8.7). The **ostream_iterator**'s second argument is used to delimit output values.

Actually, this program is longer than it needs to be. We read the strings into a **vector**, then we **sort()** them, and then we write them out, eliminating duplicates. A more elegant solution is not to store duplicates at all. This can be done by keeping the **strings** in a **set**, which does not keep duplicates and keeps its elements in order (§9.4). That way, we could replace the two lines using a **vector** with one using a **set** and replace **unique_copy()** with the simpler **copy()**:

```

    set<string> b {ii, eos};      // collect strings from input
    copy(b.begin(), b.end(), oo); // copy buffer to output

```

We used the names **ii**, **eos**, and **oo** only once, so we could further reduce the size of the program:

```

int main()
{
    string from, to;
    cin >> from >> to;           // get source and target file names

    ifstream is {from};          // input stream for file "from"
    ofstream os {to};            // output stream for file "to"

    set<string> b {istream_iterator<string>{is}, istream_iterator<string>{}}; // read input
    copy(b.begin(), b.end(), ostream_iterator<string>{os, "\n"}); // copy to output

    return !is.eof() || !os;      // return error state (§1.3, §8.4)
}

```

It is a matter of taste and experience whether or not this last simplification improves readability.

10.5 Predicates

In the examples above, the algorithms have simply “built in” the action to be done for each element of a sequence. However, we often want to make that action a parameter to the algorithm. For example, the **find** algorithm (§10.2, §10.6) provides a convenient way of looking for a specific value. A more general variant looks for an element that fulfills a specified requirement, a *predicate*. For example, we might want to search a **map** for the first value larger than **42**. A **map** allows us to access its elements as a sequence of (*key,value*) pairs, so we can search a **map<string,int>**'s sequence for a **pair<const string,int>** where the **int** is greater than **42**:

```
void f(map<string,int>& m)
{
    auto p = find_if(m.begin(),m.end(),Greater_than{42});
    // ...
}
```

Here, `Greater_than` is a function object (§5.5) holding the value (42) to be compared against:

```
struct Greater_than {
    int val;
    Greater_than(int v) : val{v} {}
    bool operator()(const pair<string,int>& r) { return r.second>val; }
};
```

Alternatively, we could use a lambda expression (§5.5):

```
auto p = find_if(m.begin(), m.end(), [](const pair<string,int>& r) { return r.second>42; });
```

A predicate should not modify the elements to which it is applied.

10.6 Algorithm Overview

A general definition of an algorithm is “a finite set of rules which gives a sequence of operations for solving a specific set of problems [and] has five important features: Finiteness ... Definiteness ... Input ... Output ... Effectiveness” [Knuth,1968,§1.1]. In the context of the C++ standard library, an algorithm is a function template operating on sequences of elements.

The standard library provides dozens of algorithms. The algorithms are defined in namespace `std` and presented in the `<algorithm>` header. These standard-library algorithms all take sequences as inputs. A half-open sequence from `b` to `e` is referred to as `[b:e)`. Here are a few examples:

Selected Standard Algorithms	
<code>p=find(b,e,x)</code>	<code>p</code> is the first <code>p</code> in <code>[b:e)</code> so that <code>*p==x</code>
<code>p=find_if(b,e,f)</code>	<code>p</code> is the first <code>p</code> in <code>[b:e)</code> so that <code>f(*p)==true</code>
<code>n=count(b,e,x)</code>	<code>n</code> is the number of elements <code>*q</code> in <code>[b:e)</code> so that <code>*q==x</code>
<code>n=count_if(b,e,f)</code>	<code>n</code> is the number of elements <code>*q</code> in <code>[b:e)</code> so that <code>f(*q,x)</code>
<code>replace(b,e,v,v2)</code>	Replace elements <code>*q</code> in <code>[b:e)</code> so that <code>*q==v</code> by <code>v2</code>
<code>replace_if(b,e,f,v2)</code>	Replace elements <code>*q</code> in <code>[b:e)</code> so that <code>f(*q)</code> by <code>v2</code>
<code>p=copy(b,e,out)</code>	Copy <code>[b:e)</code> to <code>[out:p)</code>
<code>p=copy_if(b,e,out,f)</code>	Copy elements <code>*q</code> from <code>[b:e)</code> so that <code>f(*q)</code> to <code>[out:p)</code>
<code>p=move(b,e,out)</code>	Move <code>[b:e)</code> to <code>[out:p)</code>
<code>p=unique_copy(b,e,out)</code>	Copy <code>[b:e)</code> to <code>[out:p)</code> ; don't copy adjacent duplicates
<code>sort(b,e)</code>	Sort elements of <code>[b:e)</code> using <code><</code> as the sorting criterion
<code>sort(b,e,f)</code>	Sort elements of <code>[b:e)</code> using <code>f</code> as the sorting criterion
<code>(p1,p2)=equal_range(b,e,v)</code>	<code>[p1:p2)</code> is the subsequence of the sorted sequence <code>[b:e)</code> with the value <code>v</code> ; basically a binary search for <code>v</code>
<code>p=merge(b,e,b2,e2,out)</code>	Merge two sorted sequences <code>[b:e)</code> and <code>[b2:e2)</code> into <code>[out:p)</code>

These algorithms, and many more (e.g., §12.3), can be applied to elements of containers, **strings**, and built-in arrays.

Some algorithms, such as **replace()** and **sort()**, modify element values, but no algorithm add or subtract elements of a container. The reason is that a sequence does not identify the container that holds the elements of the sequence. If you want to add elements, you need something, such as an **back_inserter** that knows about the container (§10.1), or directly refer to the container itself, such as **push_back()** or **erase()** (§9.2).

The standard-library algorithms tend to be more carefully designed, specified, and implemented than the average hand-crafted loop, so know them and use them in preference to code written in the bare language.

10.7 Container Algorithms

A sequence is defined by a pair of iterators [**begin: end**). This is general and flexible, but most often, we apply an algorithm to a sequence that is the contents of a container. For example:

```
sort(v.begin(),v.end());
```

Why don't we just say **sort(v)**? We can easily provide that shorthand:

```
namespace Estd {
    using namespace std;

    template<typename C>
    void sort(C& c)
    {
        sort(c.begin(),c.end());
    }

    template<typename C, typename Pred>
    void sort(C& c, Pred p)
    {
        sort(c.begin(),c.end(),p);
    }

    // ...
}
```

I put the container versions of **sort()** (and other algorithms) into their own namespace **Estd** ("extended **std**") to avoid interfering with other programmers' uses of namespace **std**.

10.8 Advice

- [1] The material in this chapter roughly corresponds to what is described in much greater detail in Chapter 32 of [Stroustrup,2013].
- [2] An STL algorithm operates on one or more sequences; §10.1.

- [3] An input sequence is half-open and defined by a pair of iterators; §10.1.
- [4] When searching, an algorithm usually returns the end of the input sequence to indicate “not found”; §10.2.
- [5] Algorithms do not directly add or subtract elements from their argument sequences; §10.2, §10.6.
- [6] When writing a loop, consider whether it could be expressed as a general algorithm; §10.2.
- [7] Use predicates and other function objects to give standard algorithms a wider range of meanings; §10.5, §10.6.
- [8] A predicate must not modify its argument; §10.5.
- [9] Know your standard-library algorithms and prefer them to hand-crafted loops; §10.6.
- [10] When the pair-of-iterators style becomes tedious, introduce a container/range algorithm; §10.7.